

EGL211 Project Report

Member Name	Admin No.
HOLY ANGELIN VIRSA IRWANTO	244876B
MIN THIHA	241450U
TAN YIN SIAH OLLEVYA EVALYN	244814Q
YADANAR HTUN	241451G
AWA ZILD POOT	243234R
BRANDON LIM JINHUI	232668Z

Executive Summary

This technical report aims to document the design and implementation of a highly available 3-tier web application architecture on Amazon Web Services (AWS). This project will also show the testing that was performed to ensure that all requirements of the project were met. This project will showcase core cloud computing skills and concepts which includes but is not limited to network isolation, load balancing, auto scaling, highly available databases, and the best practices to ensure high security.

Some of the key parts of this project that was implemented includes:

- The creation of a custom Virtual Private Cloud (VPC) with 6 subnets that span across 2 Availability Zones (AZs).
- Application Load Balancer (ALB) which is for distribution of traffic.
- Auto Scaling Group (ASG) to maintain having 2 application Amazon Elastic Cloud Compute (EC2) instances running at all times.
- Implement a Multi-AZ Relational Database Service (RDS) MySQL database.
- Launch a Bastion Host EC2 instance for secure administrative access.
- Create Network Address Translation (NAT) Gateways (1 for each AZ) to ensure private subnets can access the internet.
- Creation of Security Groups and Network Access Control Lists (NACLs) for defence in depth
- Elastic Block Store (EBS) Snapshots for business continuity
- CloudWatch alarms to monitor the status of the web app instances.

Contents

Executive Summary.....	2
Introduction.....	5
Background.....	5
Objectives	5
Implementation and Discussion	5
VPC and Networking (Requirement 4.1)	5
VPC Creation	5
Subnet Configuration	6
Internet Gateway	7
NAT Gateways	10
Route Tables	11
Enable Auto-assign Public IP.....	13
Security Groups (SGs)	13
Network ACLs	14
Bastion Host Configuration	15
Connecting via SSH	16
SSH to Private Instances via Bastion in PuTTY	17
Application Tier	17
Launch Template.....	17
Target Group.....	20
Database Tier - RDS MySQL	24
DB Subnet Group	24
RDS MySQL Instance.....	25
Testing Database Connectivity	26
S3 Storage Configuration	27
Production Recommendation	30
CloudWatch Monitoring	31
Testing and Validation.....	32
Web Application Access	32
High Availability Test	33
Auto Scaling Self-Healing.....	33
Bastion SSH Access.....	35
Database Connectivity.....	35
Database Not Publicly Accessible	36
Private Instance Internet Access	36

S3 Testing	36
Problem & Challenges	38
IAM Role Creation Restrictions	38
EC2 Metadata Services (IMDSv2).....	38
Bastion Host IP Change	38
Individual Reflections.....	39
Yadanar's Reflection	39
Ollevya's Reflection.....	39
Min's Reflection.....	40
Zilds's Reflection	40
Angelin's Reflection.....	41
Brandon's Reflection	41
Recommendations.....	42
Security Enhancements (AWS, 2024)	42
Cost Optimization (AWS,2024).....	43
Operational Excellence (AWS, 2024)	43
Conclusions.....	43
References	44
Appendices.....	45

Introduction

Background

Cloud computing heavily transformed how IT infrastructures are deployed (Rance, 2024). Through AWS, companies are now able to implement scalable, highly-available and cost-effective solutions. In this project, we deploy a 3-tier architecture which separates applications into application, presentation and data layers which will allow for independent scaling for improvements and efficiencies and improved security.

Objectives

In this project, we create a custom VPC with proper network segments across multiple AZs and implement proper routing with Internet Gateway (IGW) and NAT Gateways. We will also be deploying an ALB for proper traffic distribution and configure an ASG for automated management of instances.

We will also need to implement a multi-AZ RDS MySQL to ensure high availability for the database and deploy a Bastion Host for secure administrative access. We will also ensure business continuity by having Snapshots and monitoring through CloudWatch. We also ensure that we can configure S3 Buckets access through private instances without needing stored credentials.

This project will be implemented in 2 regions, AWS US East (N. Virginia) us-east-1a and us-east-1b AZs, and in these AZs will include the network, compute, database, storage and security configurations.

Implementation and Discussion

VPC and Networking (Requirement 4.1)

VPC Creation

Purpose: VPCs provide isolated network environments which ensures that the company has full control of IP addressing, subnet and routing configurations.

Configuration:

- VPC Name: proj-cas-VPC
- CIDR Block: 10.0.0.0/16 (provide 65,536 IP addresses)
- Region: US East (N. Virginia)

Procedure:

1. Create VPC → Select “VPC Only” and name the VPC “proj-cas”
2. Enter 10.0.0.0/16 as IPv4 CIDR block
3. Leave IPv6 “No IPv6 CIDR block”
4. Set the Tenancy to “Default” and click “Create VPC”

Making our IPv4 CIDR block /16 provides us with more than enough IP addresses for scalability in case of expansions. This CIDR block also ensures that we are in 1918 private address space. This ensures no conflicts with real public website’s IP, security (Gridelli, 2025).

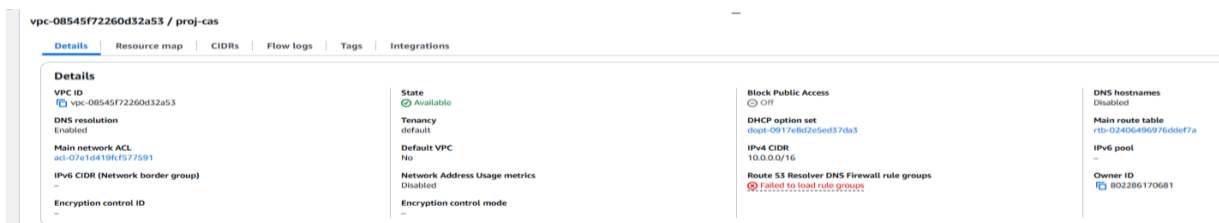


Figure 1: VPC Configuration Details

Subnet Configuration

Purpose: Subnets enable the VPC to be divided into smaller network segments. In this project, we created a total of 6 subnets, 3 in each AZ, for high availability (Senarath, 2025).

Subnet Name	CIDR Block	AZ	Type	Purpose
pub-sub-1a	10.0.1.0/24	us-east-1a	Public	ALB, Bastion, NAT GW
pub-sub-1b	10.0.2.0/24	us-east-1b	Public	ALB, NAT GW
priv-app-sub-1a	10.0.10.0/24	us-east-1a	Private	App Instances
priv-app-sub-	10.0.20.0/24	us-east-1b	Private	App Instances

1b				
priv-db-sub-1a	10.0.100.0/24	us-east-1a	Private	RDS Primary
priv-db-sub-1b	10.0.200.0/24	us-east-1b	Private	RDS Standby

Procedure:

1. Go to Subnets → Create Subnet
2. Select “proj-cas” VPC and enter the subnet name, choose the AZ, and enter the CIDR block.
3. Click “Create Subnet”
4. Repeat for all 6 subnets.

Rationale: Having subnets across multiple AZs ensures that if 1 AZ fails, it does not bring down all the applications and services. Having separate public and private subnets will help to enforce network isolation.

Internet Gateway

Purpose: An IGW allows communication between instances in the VPC and the internet for the public subnets in the VPC.

Procedure:

1. Internet Gateway → Create Internet Gateway
2. Name it “proj-igw” and click “Create Internet Gateway”
3. Select the IGW → go to “Actions” and choose “Attach to VPC”
4. Select “proj-cas” VPC and attach the IGW to it.

subnet-0770c39312229f68 / pub-sub-1a			
Details Flow logs Route table Network ACL CIDR reservations Sharing Tags			
Details Subnet ID subnet-0770c39312229f68 IPv4 CIDR 10.0.1.0/24 Availability Zone us-east-1a Network ACL acl-07e1d419f577591 Auto-assign customer-owned IPv4 address No IPv6 CIDR reservations – Resource name DNS AAAA record Disabled	Subnet ARN arn:aws:ec2:us-east-1:802286170681:subnet/subnet-0770c39312229f68 Available IPv4 addresses 249 Network border group us-east-1 Default subnet No Customer-owned IPv4 pool – IPv6-only No DNS64 Disabled	State ● Available IPv6 CIDR – VPC vpc-08545f72260d32a53 proj-cas Auto-assign public IPv4 address Yes Outpost ID – Hostname type IP name Owner 802286170681	Block Public Access ☐ Off IPv6 CIDR association ID – Route table rtb-0aa879f117ea904d pub-rtb Auto-assign IPv6 address No IPv4 CIDR reservations – Resource name DNS A record Disabled

Figure 2: Configuration for Public Subnet in us-east-1a

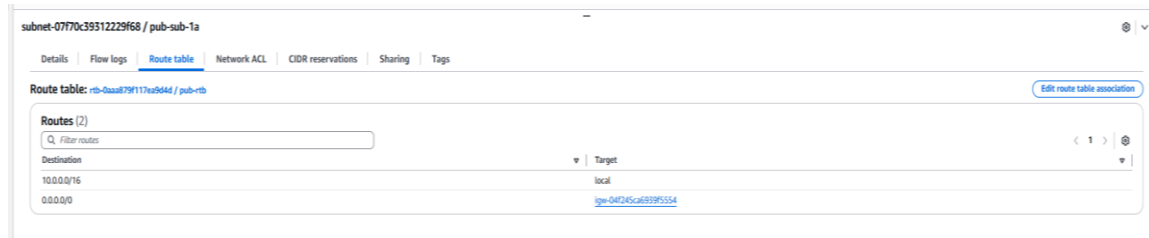


Figure 3: Shows the route table that it is associated with and the target IGW for pub-sub-1a

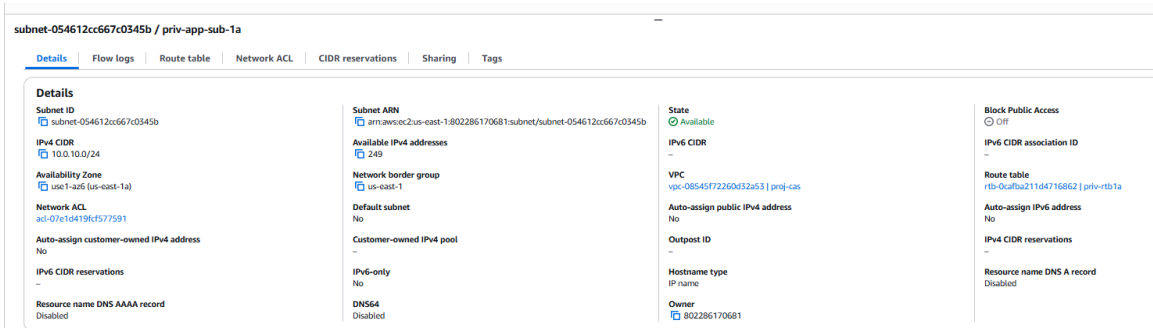


Figure 4: Configurations for private subnet in us-east-1a for Web Application

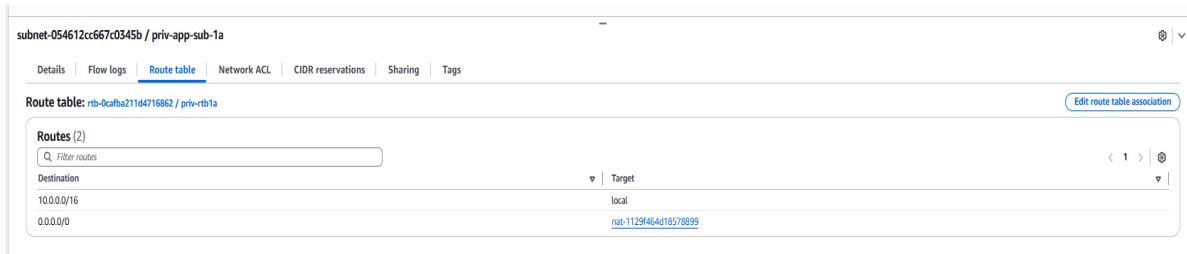


Figure 5: Shows the route table it is associated with and the target NAT Gateway for priv-app-sub-1a

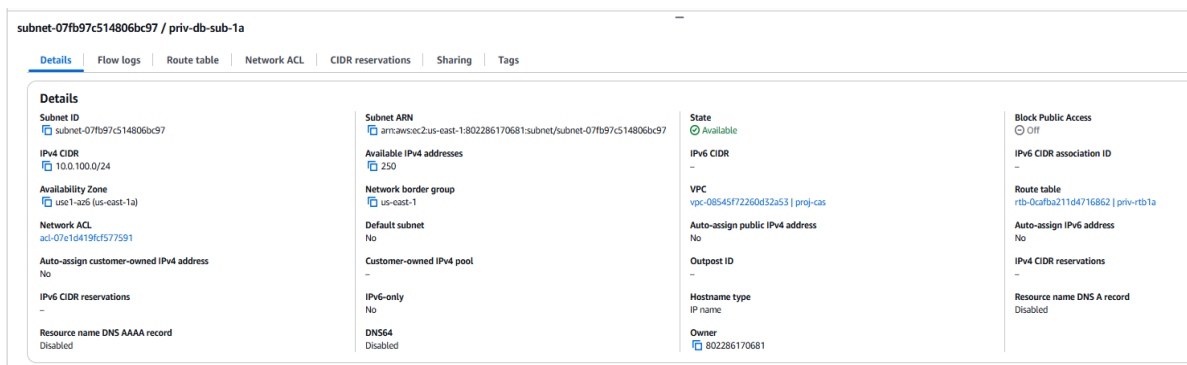


Figure 6: Configurations for Private Subnet in us-east-1a for Database

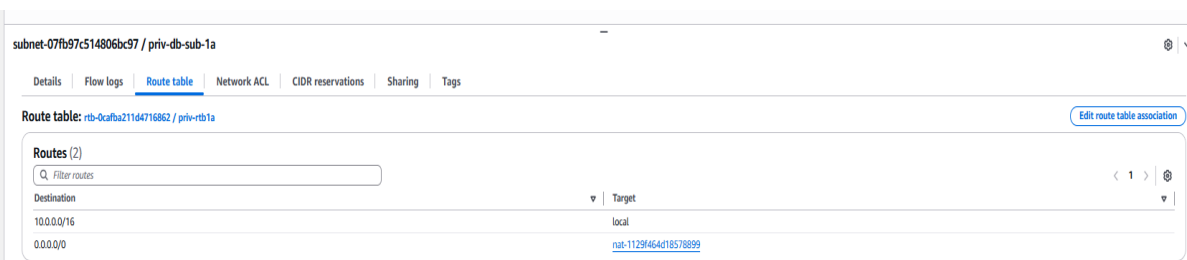


Figure 7: Shows the route table it is associated with and the target NAT Gateway for priv-db-sub-1a

subnet-057e12979339126d8 / pub-sub-1b

Details

Flow logs

Route table

Network ACL

CIDR reservations

Sharing

Tags

Details

Subnet ID

subnet-057e12979339126d8

IPv4 CIDR

10.0.2.0/24

Availability Zone

use1-az1 (us-east-1b)

Network ACL

acl-07e1d419fc577591

Auto-assign customer-owned IPv4 address

No

IPv6 CIDR reservations

-

Resource name DNS AAAA record

Disabled

Subnet ARN

arn:aws:ec2:us-east-1:802286170681:subnet/subnet-057e12979339126d8

Available IPv4 addresses

250

Network border group

us-east-1

Default subnet

No

Customer-owned IPv4 pool

-

IPv6-only

No

DNS64

Disabled

State

Available

IPv6 CIDR

-

VPC

vpc-08545f72260d32a53 | proj-cas

Auto-assign public IPv4 address

Yes

Outpost ID

-

Hostname type

IP name

Owner

802286170681

Block Public Access

Off

IPv6 CIDR association ID

-

Route table

rtb-0aaa879f117ea9d4d | pub-rtb

Auto-assign IPv6 address

No

IPv4 CIDR reservations

-

Resource name DNS A record

Disabled

State

Available

IPv6 CIDR

-

VPC

vpc-08545f72260d32a53 | proj-cas

Auto-assign public IPv4 address

Yes

Outpost ID

-

Hostname type

IP name

Owner

802286170681

Block Public Access

Off

IPv6 CIDR association ID

-

Route table

rtb-0aaa879f117ea9d4d | pub-rtb

Auto-assign IPv6 address

No

IPv4 CIDR reservations

-

Resource name DNS A record

Disabled

Figure 8: Configurations for Public Subnet in us-east-1b

subnet-057e12979339126d8 / pub-sub-1b							
Details Flow logs Route table Network ACL CIDR reservations Sharing Tags							
Route table: rtb-0aaa879f117ea9d4d / pub-rtb							
Routes (2) Filter routes <table> <thead> <tr> <th>Destination</th><th>Target</th></tr> </thead> <tbody> <tr> <td>10.0.0.0/16</td><td>local</td></tr> <tr> <td>0.0.0.0/0</td><td>igw-04f245ca6839f5554</td></tr> </tbody> </table>		Destination	Target	10.0.0.0/16	local	0.0.0.0/0	igw-04f245ca6839f5554
Destination	Target						
10.0.0.0/16	local						
0.0.0.0/0	igw-04f245ca6839f5554						

Figure 9: Shows the route table it is associated with and the target IGW for pub-sub-1b

subnet-01a57939cc30b81f7 / priv-app-sub-1b

Details

Flow logs

Route table

Network ACL

CIDR reservations

Sharing

Tags

Details

Subnet ID

subnet-01a57939cc30b81f7

IPv4 CIDR

10.0.20.0/24

Availability Zone

use1-az1 (us-east-1b)

Network ACL

acl-07e1d419fc577591

Auto-assign customer-owned IPv4 address

No

IPv6 CIDR reservations

-

Resource name DNS AAAA record

Disabled

Subnet ARN

arn:aws:ec2:us-east-1:802286170681:subnet/subnet-01a57939cc30b81f7

Available IPv4 addresses

250

Network border group

us-east-1

Default subnet

No

Customer-owned IPv4 pool

-

IPv6-only

No

DNS64

Disabled

State

Available

IPv6 CIDR

-

VPC

vpc-08545f72260d32a53 | proj-cas

Auto-assign public IPv4 address

No

Outpost ID

-

Hostname type

IP name

Owner

802286170681

Block Public Access

Off

IPv6 CIDR association ID

-

Route table

rtb-0d5a8ae82f6c27b3 | priv-rtb-1b

Auto-assign IPv6 address

No

IPv4 CIDR reservations

-

Resource name DNS A record

Disabled

State

Available

IPv6 CIDR

-

VPC

vpc-08545f72260d32a53 | proj-cas

Auto-assign public IPv4 address

No

Outpost ID

-

Hostname type

IP name

Owner

802286170681

Block Public Access

Off

IPv6 CIDR association ID

-

Route table

rtb-0d9a8ae82fd6c2fb3 | priv-rtb-1b

Auto-assign IPv6 address

No

IPv4 CIDR reservations

-

Resource name DNS A record

Disabled

Figure 10: Configurations for Private Subnet in us-east-1b for Web Application

subnet-01a57939cc30b81f7 / priv-app-sub-1b							
Details Flow logs Route table Network ACL CIDR reservations Sharing Tags							
Route table: rtb-0d9a8ae82fd6c2fb3 / priv-rtb-1b							
Routes (2) Filter routes <table> <thead> <tr> <th>Destination</th><th>Target</th></tr> </thead> <tbody> <tr> <td>10.0.0.0/16</td><td>local</td></tr> <tr> <td>0.0.0.0/0</td><td>nat-16f6a9151b13ec784</td></tr> </tbody> </table>		Destination	Target	10.0.0.0/16	local	0.0.0.0/0	nat-16f6a9151b13ec784
Destination	Target						
10.0.0.0/16	local						
0.0.0.0/0	nat-16f6a9151b13ec784						

Figure 11: Shows route tables associated with the private subnet and the targeted NAT gateway

subnet-0bd0530c95cfd605e / priv-db-sub-1b			
Details Flow logs Route table Network ACL CIDR reservations Sharing Tags			
Details			
Subnet ID subnet-0bd0530c95cfd605e	Subnet ARN arn:aws:ec2:us-east-1:802286170681:subnet/subnet-0bd0530c95cfd605e	State Available	Block Public Access Off
IPv4 CIDR 10.0.200.0/24	Available IPv4 addresses 250	IPv6 CIDR -	IPv6 CIDR association ID -
Availability Zone us-east-1b (us-east-1b)	Network border group us-east-1	VPC vpc-08545f72260d32a53 proj-cas	Route table rtb-0d9a8ae82fd6c2fb3 priv-rtb-1b
Network ACL acl-07e16419cf577591	Default subnet No	Auto-assign public IPv4 address No	Auto-assign IPv6 address No
Auto-assign customer-owned IPv4 address No	Customer-owned IPv4 pool -	Outpost ID -	IPv4 CIDR reservations -
IPv6 CIDR reservations -	IPv6-only No	Hostname type IP name	Resource name DNS A record Disabled
Resource name DNS AAAA record Disabled	DNS64 Disabled	Owner 802286170681	

Figure 12: Configuration for Private Subnet in us-east-1b for Database

subnet-0bd0530c95cfd605e / priv-db-sub-1b	
Details Flow logs Route table Network ACL CIDR reservations Sharing Tags	
Route table: rtb-0d9a8ae82fd6c2fb3 / priv-rtb-1b	
Routes (2)	
Destination	Target
10.0.0.0/16	local
0.0.0.0/0	nat-16f6a9151b13ec784

Figure 13: Shows the route tables associated with the subnet and the target NAT Gateway it is routed towards

NAT Gateways

Purpose: This will allow instances with private subnets to access the internet for patching and updating purposes while still being inaccessible from the internet.

Why did we make 2 NAT Gateways:

- If we have a single NAT Gateway, this would mean that if the NAT fails to connect to the internet, all of our private subnets will not be able to access the internet for patching and updates.
- Having 2 NAT gateways (one in each AZ) will allow for high availability for regular patching and updates.

Procedure:

1. NAT Gateways → Create NAT Gateway
2. Names: “proj-nat-1a-gw” and “proj-nat-1b-gw”
3. Subnet: “pub-sub-1a” and “pub-sub-1b” respectively
4. Connectivity type: “Public”
5. “Allocate Elastic IP” and then proceed to press “Create NAT Gateway”
6. Ensure that the status becomes “Available”

nat-1129f464d18578899 / proj-nat1a-gw			
Details	IP addresses	Monitoring	Flow logs
Tags			
Details			
NAT gateway ID nat-1129f464d18578899	Availability mode Regional	State Available	State message -
NAT gateway ARN arn:aws:ec2:us-east-1:802286170681:natgateway/nat-1129f464d18578899	Connectivity type Public	Created Wednesday, February 11, 2026 at 14:07:33 GMT+8	Deleted -
VPC vpc-08545f72260d32a53 / proj-cas	Method of EIP allocation Automatic		

Figure 14: Shows the NAT Gateway Configurations for us-east-1a

nat-1129f464d18578899 / proj-nat1a-gw

Details

IP addresses

Monitoring

Flow logs

Tags

Associated IP addresses (2)

Q Search

IP address	Status	Availability Zone	Allocation ID	Association ID
35.172.24.10	Succeeded	use1-az6 (us-east-1a)	eipalloc-07b451eacb090b841	eipassoc-0a06df5c960723384
100.30.246.38	Succeeded	use1-az1 (us-east-1b)	eipalloc-083a21a2fc0d61470	eipassoc-050d89a90d27fa504

Figure 15: Shows the subnets associated with the NAT Gateway

nat-16f6a9151b13ec784 / proj-nat1b-gw			
Details	IP addresses	Monitoring	Flow logs
Tags			
Details			
NAT gateway ID nat-16f6a9151b13ec784	Availability mode Regional	State Available	State message -
NAT gateway ARN arn:aws:ec2:us-east-1:802286170681:natgateway/nat-16f6a9151b13ec784	Connectivity type Public	Created Thursday, February 12, 2026 at 14:53:42 GMT+8	Deleted -
VPC vpc-08545f72260d32a53 / proj-cas	Method of EIP allocation Automatic		

Figure 16: Shows the NAT Gateway Configurations for us-east-1b

nat-16f6a9151b13ec784 / proj-nat1b-gw

Details

IP addresses

Monitoring

Flow logs

Tags

Associated IP addresses (2)

Q Search

IP address	Status	Availability Zone	Allocation ID	Association ID
52.207.202.82	Succeeded	use1-az1 (us-east-1b)	eipalloc-0fc44b71026c1c90a	eipassoc-084b7d6b84bf3a27
107.23.61.197	Succeeded	use1-az6 (us-east-1a)	eipalloc-0eb040ce6fd6dd1a92	eipassoc-0c8e30177b5a51c3e

Figure 17: Shows the subnets Associated with the NAT Gateway

Route Tables

Purpose: Route tables will help to determine where the network traffic is being directed to.

Route Table	Destination	Target	Associated Subnets
pub-rtb	0.0.0.0/0	IGW	pub-sub-1a, pub-sub-1b
priv-rtb-1a	0.0.0.0/0	nat-gw-1a	priv-app-sub-1a, priv-db-sub-1a
priv-rtb-1b	0.0.0.0/0	nat-gw-1b	Priv-app-sub-1b, priv-db-sub-1b

Procedure for Public RTB:

1. Route Tables → Create Route Table
2. Name: “pub-rtb” and select “proj-cas” as the VPC and click “Create Route Table”
3. Select “pub-rtb” go to the Routes tab and edit routes
4. Add route: 0.0.0.0/0 and target it towards “proj-igw” and save changes
5. Subnet associations → edit and select both public subnets in the 2 AZs and save

Procedure for Private RTBs:

1. Create “priv-rtb-1a” and “priv-rtb-1b”, add a route to 0.0.0.0/0 and target them towards “nat-gw-1a” and “nat-gw-1b” respectively.
2. For priv-rtb-1a associate private subnets from us-east-1a
3. For priv-rtb-1b associate private subnets from us-east-1b

Rationale: Private subnets in their respective AZs are routed through their own NAT GW, so that if one fails, the subnets in the other AZs can still have internet access.

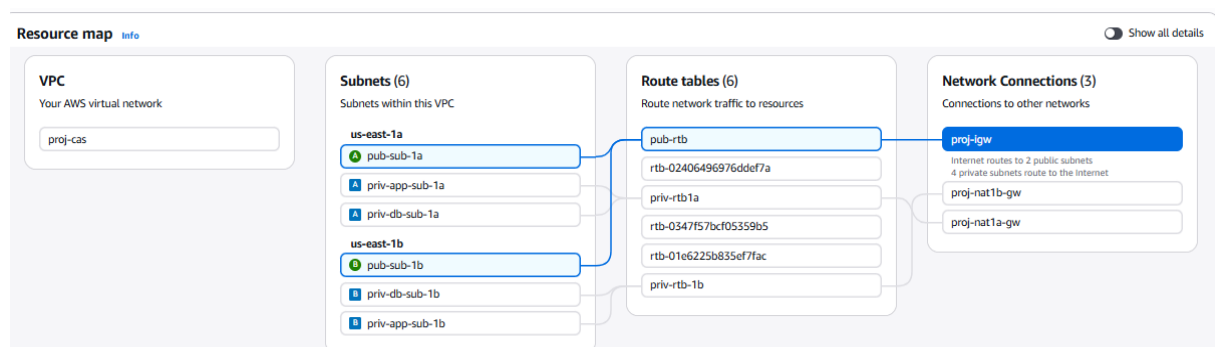


Figure 18: Resource map in VPC showing where public subnets routing towards proj-igw

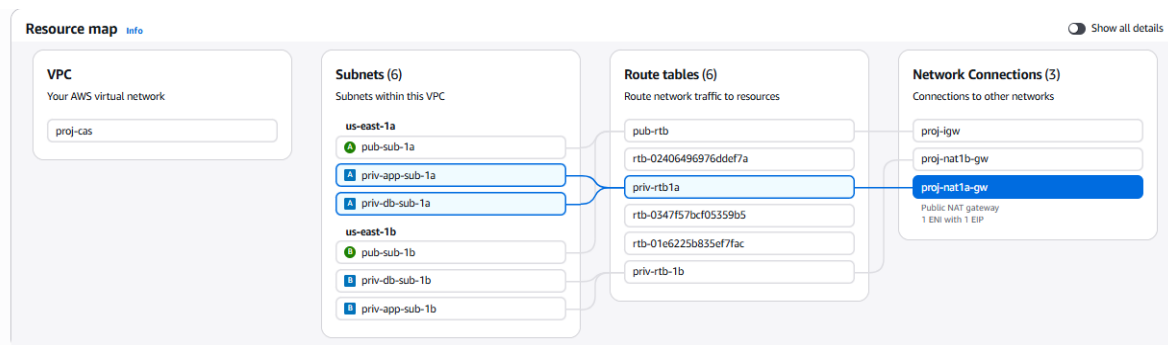


Figure 19: Resource map in VPC showing private subnets in us-east-1a route towards proj-nat-1a-gw

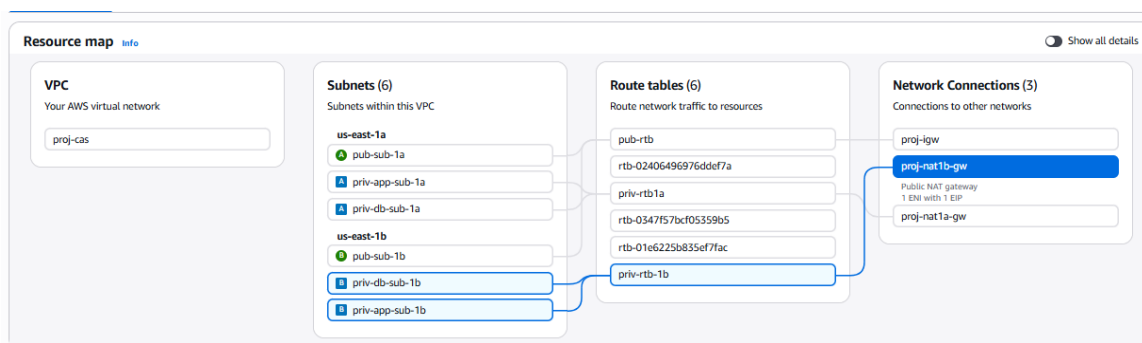


Figure 20: Resource map in VPC showing private subnets in us-east-1b routing towards proj-nat-1b-gw

Enable Auto-assign Public IP

Purpose: Instances in public subnets need public IPs for internet accessibility.

Procedure:

1. Go to Subnets and select “pub-sub-1a”
2. Click “Actions” and proceed to edit subnet settings
3. Check “Enable auto-assign public IPv4 address” and save
4. Repeat for “pub-sub-1b”

Security Groups (SGs)

Purpose: This acts as virtual firewalls at instance level and are stateful. This means that return traffic is automatically allowed. Outbound rules remain the same for all SGs, which allow traffic from anywhere at IPv4.

Security Group	Inbound Rules	Purpose
alb-sg	HTTP(80), HTTPS(443) from	Allow web traffic to ALB

	0.0.0.0/0	
bastion-sg	SSH(22) from 0.0.0.0/0	Allows SSH to Bastion only
app-sg	HTTP(80) from alb-sg, SSH(22) from bastion-sg	App accepts from ALB and Bastion only
db-sg	MySQL(3306) from app-sg	DB accepts from App only

Procedure for SG Creation:

1. Go to Security Groups → Create Security Group
2. Give the security group a name and a description Eg: “alb-sg”, “Security group for ALB”
3. Connect to “proj-cas” VPC
4. Add the inbound, Eg: Type: HTTP or SSH, Source: 0.0.0.0/0 or bastion-sg (follow the table above)
5. Leave the outbound rules as default (Allow all)
6. Click “Create Security Group”

Rationale: This follows the principle of least privilege, where each part only accepts traffic from approved sources.

Network ACLs

Purpose: ACLs are subnet-level firewalls which are stateless and hence you must explicitly allow return traffic.

Private-NACL Inbound Rules:

Rule #	Type	Port	Source	Allow/Deny	Purpose
100	HTTP	80	10.0.0.0/16	Allow	Web traffic from VPC
110	SSH	22	10.0.0.0/16	Allow	Admin access from VPC

*	All	All	0.0.0.0/0	Deny	Default Deny
---	-----	-----	-----------	------	--------------

Private-NACL Outbound Rules:

Rule #	Type	Port	Source	Allow/Deny	Purpose
100	HTTP	80	0.0.0.0/16	Allow	Outbound web
110	HTTPS	443	0.0.0.0/16	Allow	Outbound secure web
*	All	All	0.0.0.0/0	Deny	Default Deny

Procedure:

1. Go to Network ACLs → Create network ACL
2. Name: “priv-nacl”, and select “proj-cas” as the VPC
3. Create and select it
4. Edit the inbound and outbound rules
5. Subnet associations → edit and select all 4 private subnets and save

Bastion Host Configuration

Purpose: The Bastion host allows a single secure entry point for SSH access to private instances. This access point acts as where the traffic for administrative matters will flow.

Bastion Host Deployment

Procedure:

1. EC2 Console → Launch instances
2. Name: “Bastion-Host”
3. AMI: Amazon Linux 2023 AMI
4. Instance type: t2.micro
5. Key pair: “vockey”

6. Network settings → Edit:

- VPC: proj-cas
- Subnet: pub-sub-1a
- Auto-assign public IP: Enable
- Security group: Bastion-SG

7. Launch instance

Instance summary for i-04bcdafb25e82304b (bastion host) [Info](#)

Updated less than a minute ago

Instance ID i-04bcdafb25e82304b	Public IPv4 address 54.83.72.118 open address	Private IPv4 addresses 10.0.1.247
IPv6 address -	Instance state Running	Public DNS -
Hostname type IP name: ip-10-0-1-247.ec2.internal	Private IP DNS name (IPv4 only) ip-10-0-1-247.ec2.internal	Elastic IP addresses 54.83.72.118 [Public IP]
Answer private resource DNS name -	Instance type t2.micro	AWS Compute Optimizer finding Opt-in to AWS Compute Optimizer for recommendations. Learn more
Auto-assigned IP address -	VPC ID vpc-08545f72260d32a53 (proj-cas)	Auto Scaling Group name -
IAM role -	Subnet ID subnet-07f70c39312229f68 (pub-sub-1a)	Managed false
IMDSv2 Required	Instance ARN arn:aws:ec2:us-east-1:802286170681:instance/i-04bcdafb25e82304b	
Operator -		

Details | Status and alarms | Monitoring | Security | Networking | Storage | Tags

▼ Instance details [Info](#)

AMI ID ami-0c1fe732b5494dc14	Monitoring disabled	Platform details Linux/UNIX
AMI name al2023-ami-2023.10.20260202.2-kernel-6.1-x86_64	Allowed image -	Termination protection Disabled
Stop protection Disabled	Launch time Fri Feb 13 2026 00:12:12 GMT+0800 (Singapore Standard Time) (about 1 hour)	AMI location amazon/al2023-ami-2023.10.20260202.2-kernel-6.1-x86_64
Instance reboot migration Default (On)	Instance auto-recovery Default	Lifecycle normal
Stop-hibernate behavior Disabled	AMI Launch index 0	Key pair assigned at launch vockey
State transition reason -	Credit specification standard	Kernel ID -

Figure 21: Bastion Host Instance Configurations and Details

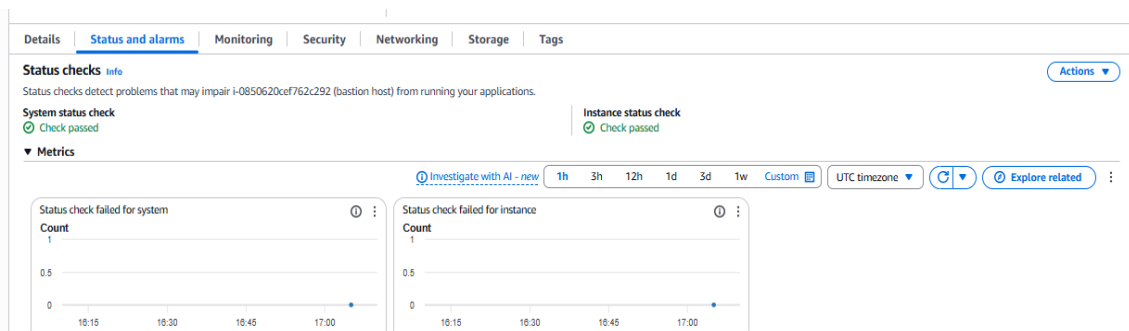


Figure 22: Bastion Host Status and Alarms Tab

Connecting via SSH

From Windows (PuTTY):

1. Open PuTTY, enter: ec2-user@<Bastion-public-IP>
2. Connection → SSH → Auth → Credentials → Browse for .ppk file
3. Click Open

SSH to Private Instances via Bastion in PuTTY

Procedure:

1. Copy Bastion's public IP address and attach it in PuTTY as "ec2-user@<Bastion-Public-IP>"
2. Open Pageant (PuTTY authentication agent), click "Add Key", select "labuser.ppk" file from its location, and then close the window once the key has been added successfully.
3. In Auth check "allow agent forwarding".
4. In credentials ensure to browse and use the "labuser.ppk" file before clicking open.
5. Once a bird appears, type "ssh ec2-user@<PrivateInstance-Private-IP>"
6. If another bird appears, it proves that the connection is successful.

```
ec2-user@ip-10-0-10-166:~  
Using username "ec2-user".  
Authenticating with public key "imported-openssh-key"  
  
#  
~\##### Amazon Linux 2023  
~~\#####\  
~~\###|  
~~\#/ https://aws.amazon.com/linux/amazon-linux-2023  
~~V~' ~->  
~~~~  
~~~~.  
~/m/' ~-/  
Last login: Thu Feb 12 17:43:06 2026 from 116.14.82.234  
[ec2-user@ip-10-0-1-91 ~]$ ssh ec2-user@10.0.10.166  
The authenticity of host '10.0.10.166 (10.0.10.166)' can't be established.  
ED25519 key fingerprint is SHA256:4eCDXycdcjq7lOim8vdVyQmnt4ISRrOWukE+GoHqKU.  
This key is not known by any other names  
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes  
Warning: Permanently added '10.0.10.166' (ED25519) to the list of known hosts.  
ec2-user@10.0.10.166: Permission denied (publickey,gssapi-keyex,gssapi-with-mic)  
  
[ec2-user@ip-10-0-1-91 ~]$ ssh ec2-user@10.0.10.166  
  
#  
~\##### Amazon Linux 2023  
~~\#####\  
~~\###|  
~~\#/ https://aws.amazon.com/linux/amazon-linux-2023  
~~V~' ~->  
~~~~  
~~~~.  
~/m/' ~-/  
[ec2-user@ip-10-0-10-166 ~]$
```

Figure 23: PuTTY showing that Bastion Host Able to Access to Private App Instance

Application Tier

Launch Template

Purpose: Launch Templates help to define configurations for instances that are launched by Auto Scaling.

Configuration:

- Name: app-launch-template

- AMI: Amazon Linux 2023
- Instance Type: t2.micro
- Key Pair: vockey
- Security Group: app-sg

User Data:

```
#!/bin/bash
dnf update -y
dnf install -y httpd mariadb105
systemctl start httpd
systemctl enable httpd

TOKEN=$(curl -X PUT "http://169.254.169.254/latest/api/token" -H "X-aws-ec2-
metadata-token-ttl-seconds: 21600")
INSTANCE_ID=$(curl -s -H "X-aws-ec2-metadata-token: $TOKEN"
http://169.254.169.254/latest/meta-data/instance-id)
AZ=$(curl -s -H "X-aws-ec2-metadata-token: $TOKEN"
http://169.254.169.254/latest/meta-data/placement/availability-zone)

cat > /var/www/html/index.html << EOF
<!DOCTYPE html>
<html>
<head>
  <title>App Server</title>
  <style>
    body { font-family: Arial; text-align: center; padding: 50px; }
    .box { background: #f0f0f0; padding: 20px; margin: 20px auto; max-width:
400px; border-radius: 10px; }
  </style>
</head>
<body>
  <h1>Application Server</h1>
  <div class="box">
    <p><strong>Instance ID:</strong> $INSTANCE_ID</p>
```

```
<p><strong>Availability Zone:</strong> $AZ</p>
</div>
</body>
</html>
EOF
```

Script Explanation:

- Updates system packages
- Installs Apache web server and MySQL client and enables apache to start on boot
- Uses IMDSv2, which are token-based, for metadata access and creates a web page that displays instance ID and AZ for the demonstration

Procedure:

1. EC2 Console → Launch Templates → Create launch template
2. Name: “App-Launch-Template”
3. AMI: Amazon Linux 2023
4. Instance type: t2.micro
5. Key pair: vockey
6. Network settings → Don’t include subnet; Security group: app-sg
7. Advanced details → User data: Paste user data script
8. Create launch template

The screenshot displays the AWS Management Console interface for a launch template named 'app-launch-temp' (ID: lt-01f036f11e519658c). The interface is divided into two main sections: 'Launch template details' and 'Launch template version details'.

Launch template details:

- Launch template ID:** lt-01f036f11e519658c
- Launch template name:** app-launch-temp
- Default version:** 2
- Owner:** arn:aws:sts:802286170681:assumed-role/vocilabs/user4545456-2448768@mymail.nyp.edu.sg

Launch template version details (Version 2 - Default):

- Description:** -
- Date created:** 2026-02-11T07:29:23.000Z
- Created by:** arn:aws:sts:802286170681:assumed-role/vocilabs/user4545456-2448768@mymail.nyp.edu.sg

Instance details:

- AMI ID:** ami-0c1fe732b5494dc14
- Instance type:** t2.micro
- Availability Zone:** -
- Availability Zone Id:** -
- Key pair name:** vockey
- Security groups:** sg-00cad972ba9e68907

Figure 24: App Launch Template Configurations

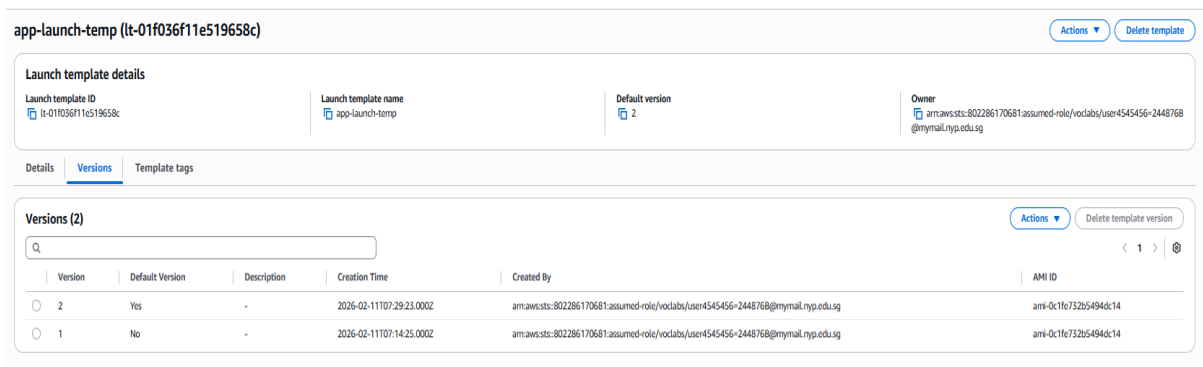


Figure 25: App Launch Template Versions, to showcase ability to update instance configurations

Target Group

Purpose: This directs the load balancer on where to target the traffic to and shows if instances are healthy or unhealthy.

Configuration:

- Name: app-tg
- Target Type: Instances
- Protocol: HTTP, Port: 80
- VPC: proj-cas
- Health check path: /
- Healthy threshold: 2
- Unhealthy threshold: 2
- Timeout: 5 seconds
- Interval: 30 seconds

Procedure:

1. EC2 Console → Target Groups → Create target group
2. Target type: Instances
3. Name: “app-tg”, Protocol: HTTP, Port: 80
4. Choose “proj-cas” as VPC
5. Health check path: /
6. Click Next → Create (do not register targets)

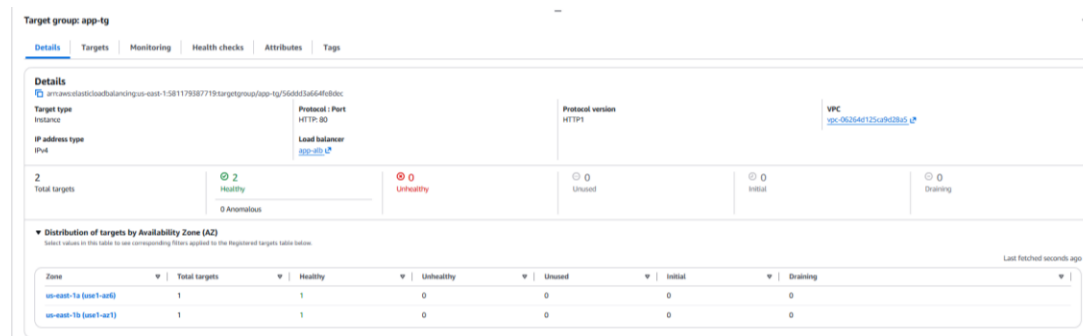


Figure 26: Target Group Details and Configurations

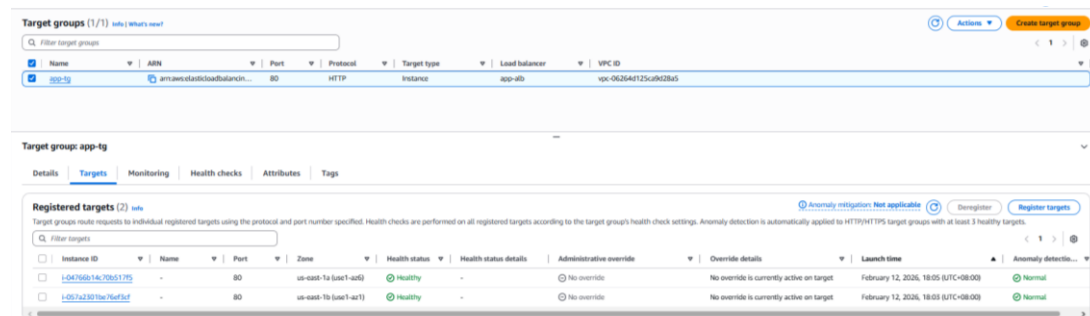


Figure 27: Target Group Targets showing "Healthy" health status

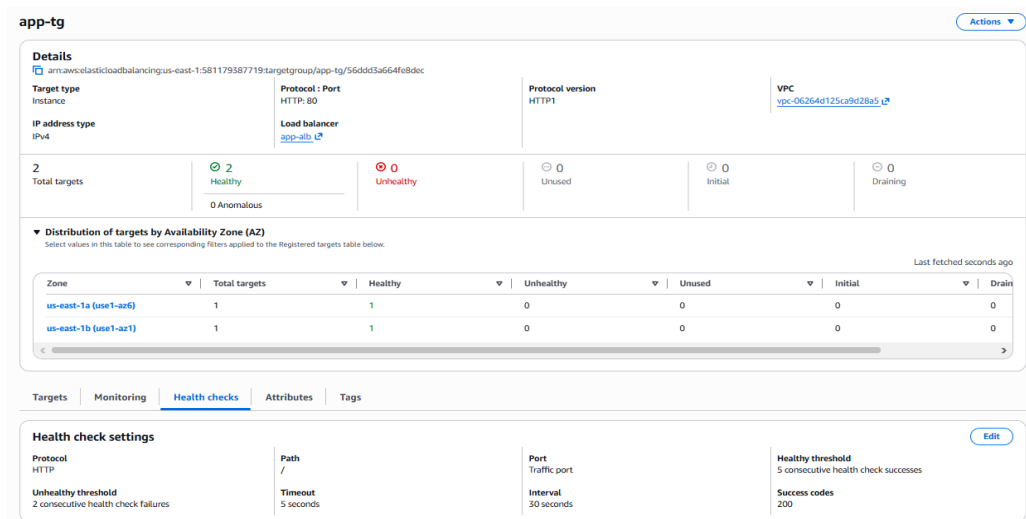


Figure 28: Target Group Health Checks Configurations

Application Load Balancer

Purpose: ALB helps to distribute inbound traffic across the instances in multiple AZs.

Configuration:

- Name: app-alb
- Scheme: Internet-facing
- IP type: IPv4
- VPC: proj-cas
- Subnets: pub-sub-1a, pub-sub-1b

- Security Groups: alb-sg
- Listener: HTTP: 80 → Forward to app-tg

Procedure:

1. EC2 Console → Load Balancers → Create Load Balancer
2. Select Application Load Balancer
3. Name: “app-alb”, Scheme: Internet-facing
4. Network mapping: VPC: proj-cas
5. Select both us-east-1a (pub-sub-1a) and us-east-1b (pub-sub-1b)
6. Security groups: alb-sg
7. Listener: HTTP:80 → Forward to app-tg
8. Create Load Balancer

The screenshot displays the AWS Management Console for an Application Load Balancer named 'app-alb'. The 'Details' tab is active, showing the load balancer's status as 'Active', its VPC (vpc-06264d125ca9d28a5), and its availability zones (us-east-1b and us-east-1a). The 'Listeners and rules' tab is also visible, showing a single listener for HTTP:80 that forwards traffic to the target group 'app-tg'.

Figure 29: ALB Details and Listeners and Rules

The screenshot displays the AWS Management Console for the 'app-alb' load balancer, specifically the 'Network mapping' and 'Availability Zones and subnets' tabs. The 'Network mapping' tab shows the VPC (vpc-06264d125ca9d28a5) and the load balancer's IP address type (IPv4). The 'Availability Zones and subnets' tab shows the load balancer's IP address type (IPv4) and the subnets assigned to it (subnet-07698421a3d9df945 and subnet-04f18321110d47c8d).

Figure 30: ALB Network Mapping

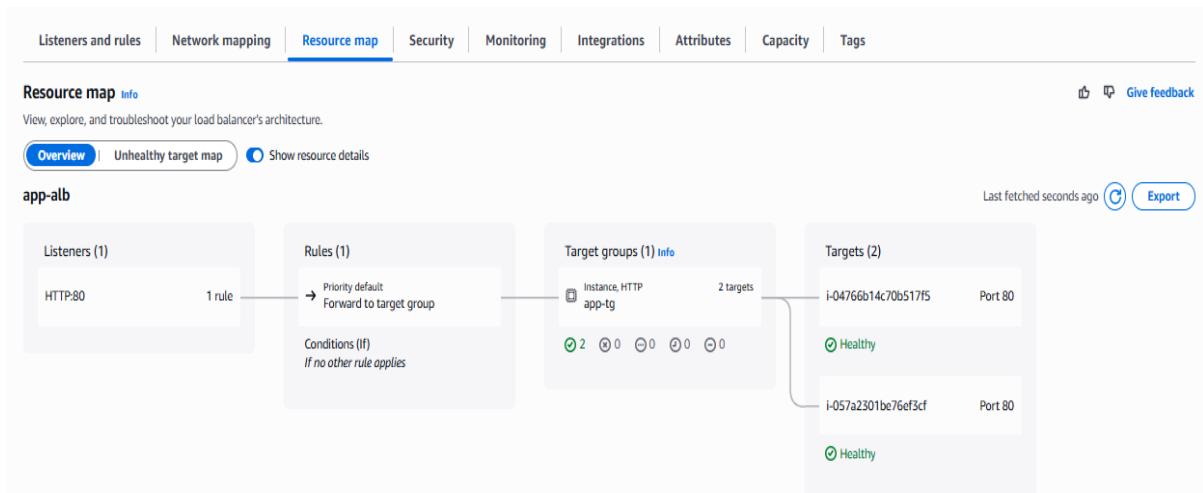


Figure 31: ALB Resource Map

The screenshot shows the 'Security' tab for an ALB. It contains a table of security groups. The table has columns for 'Security Group ID', 'Name', and 'Description'. One entry is shown: 'sg-021ea8db2fd5581e9' with name 'alb-sg' and description 'security group for alb'.

Security Group ID	Name	Description
sg-021ea8db2fd5581e9	alb-sg	security group for alb

Figure 32: ALB Security Tab

Auto Scaling Group

Purpose: This automatically ensures that the minimum desired number of instances is constantly available. Thus, replacing any unhealthy instances that may occur.

Configuration:

- Name: app-asg
- Launch template: app-launch-template
- VPC: proj-cas
- Subnets: priv-app-sub-1a, priv-app-sub-1b
- Load Balancer: Attached to app-tg
- Desired/Minimum: 2
- Maximum: 4

Procedure:

1. Go to Auto Scaling Groups and create an ASG
2. Name it “app-asg” and choose “app-launch-template”, select “proj-cas” as VPC
3. Subnets: “priv-app-sub-1a” and “priv-app-sub1b”
4. Load balancing: attach to existing load balancer and choose “app-tg”
5. Check “Turn on Elastic Load Balancing Health Checks”

6. Desired: 2, Minimum: 2, Maximum: 4

7. Create Auto Scaling Group

The screenshot displays the 'app-asg' Auto Scaling Group configuration in the AWS Management Console. The interface includes a top navigation bar with tabs for 'Details', 'Integrations', 'Automatic scaling', 'Instance management', 'Instance refresh', 'Activity', 'Monitoring', and 'Tags - moved'. The main content area is divided into several sections:

- app-asg Capacity overview:** Shows the group's ARN, desired capacity (2), scaling limits (2-4), desired capacity type (Units (number of instances)), and status (Ready). It also indicates the date created (Thu Feb 12 2026 00:01:04 GMT+0800 (Singapore Standard Time)).
- Launch template:** Details the launch template (lt-02824ba67965580f2), AMI ID (ami-0c1e732b5494dc14), instance type (t2.micro), security groups, security group IDs, key pair name (vockey), and owner information.
- Network:** Shows the availability zones (us-east-1a, us-east-1b), subnet IDs (subnet-088497fa52ac7153c, subnet-052f7b3ee73819e9), and availability zone distribution (Balanced best effort).
- Instance type requirements:** States that the Auto Scaling group adheres to the launch template for purchase option and instance type.
- Health checks:** Shows the health check type (EC2, ELB) and health check grace period (300).

Figure 33: ASG Configurations and Details

Database Tier - RDS MySQL

DB Subnet Group

Purpose: This helps to define which subnets the RDS can use to deploy.

Procedure:

1. RDS Console → Subnet groups → Create DB subnet group
2. Name: "proj-db-sub-grp"
3. Choose "proj-cas" as VPC
4. Add subnets:
 - AZ: us-east-1a → Priv-db-sub-1a
 - AZ: us-east-1b → Priv-db-sub-1b
5. Create

proj-db-sub-grp			
Subnet group details			
VPC ID			
vpc-06264d125ca9d28a5 ↗			
ARN			
arn:aws:rds:us-east-1:581179387719:subgrp:proj-db-sub-grp			
Supported network types			
IPv4			
Description			
db subnet group			
Subnets (2)			
Availability zone	Subnet name	Subnet ID	CIDR block
us-east-1b	pub-sub-1b	subnet-07698421a3d9df945 ↗	10.0.2.0/24
us-east-1a	pub-sub-1a	subnet-04f18321110d47c8d ↗	10.0.1.0/24

Figure 34: Database Subnet Group Details

RDS MySQL Instance

Configuration:

- Engine: MySQL 8.0
- Template: Dev/Test (for Multi-AZ)
- DB Instance ID: project-mysql-db
- Master Username: admin
- Instance Class: db.t3.micro (burstable)
- Storage: 20GB gp3
- Multi-AZ: Yes (standby in different AZ)
- VPC: proj-cas
- Subnet Group: proj-db-sub-grp
- Public Access: No
- Security Group: db-sg

Procedure:

1. RDS Console → Databases → Create database
2. Choose Standard Create and MySQL the default version as engine
3. Template: Dev/Test and name the db instance identifier as “proj-mysql-db”
4. Master username: admin

5. Master password: hello123
6. Instance Class: db.t3.micro (Choose burstable)
7. Storage: 20GB, gp3, disable autoscaling
8. Multi-AZ deployment: Create a standby instance
9. VPC: "proj-cas"
10. Subnet Group: proj-db-sub-grp
11. Public access: no
12. Security Group: db-sg and remember to remove default-sg
13. Click create database and wait for it to have "Available" status which can take up to 15 minutes.

Rationale: Deploying the RDS in multiple AZs will provide us with automatic failover, so if the primary AZ fails, the standby is promoted to be the primary within 60 seconds. Public access should be disabled for security purposes and only allow app-sg to connect to the RDS through private IP.

Testing Database Connectivity

From Bastion Host:

```
sudo dnf install mariadb105 -y
mysql -h <RDS-ENDPOINT> -u admin -p
```

Expected Results: It should not be successful as the database should only be accessible through private IP through the App Instances.

```
ec2-user@ip-10-0-1-91:~  
Using username "ec2-user".  
Authenticating with public key "imported-openssh-key" from agent  
  
#  
~\#####  
~\#####  
~\####|  
~\#/ https://aws.amazon.com/linux/amazon-linux-2023  
~V~' ->  
~~~  
~-.-  
~/m/'-  
  
Last login: Fri Feb 13 05:38:20 2026 from 210.10.76.120  
[ec2-user@ip-10-0-1-91 ~]$ mysql -h proj-mysql-db.cznzajrpojxx.us-east-1.rds.amazonaws.com -u admin -p  
Enter password:  
ERROR 2002 (HY000): Can't connect to MySQL server on 'proj-mysql-db.cznzajrpojxx.us-east-1.rds.amazonaws.com' (115)  
[ec2-user@ip-10-0-1-91 ~]$
```

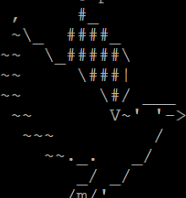
Figure 35: Bastion (Public IP address) trying to access into RDS but Failed

From App Instance (via Bastion):

```
Mysql -h <RDS-ENDPOINT> -u admin -p
```

Expected Results: Successful connection should be seen.

```
[ec2-user@ip-10-0-1-91 ~]$ ssh ec2-user@10.0.10.166
```



```
      #####  
    ~\_____/#####  
    ~\_____/\###|  
    ~\_____\  \#/   https://aws.amazon.com/linux/amazon-linux-2023  
    ~~~~~V~' ->  
    ~~~~.  
    ~~~~_._  
    ~~~~/_m/'
```

```
Amazon Linux 2023
```

```
[ec2-user@ip-10-0-10-166 ~]$ mysql -h proj-mysql-db.cznzajrpojxx.us-east-1.rds.amazonaws.com -u admin -p
```

```
Enter password:  
Welcome to the MariaDB monitor. Commands end with ; or \g.  
Your MySQL connection id is 282  
Server version: 8.4.7 Source distribution
```

```
Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.
```

```
Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.
```

```
MySQL [(none)]> █
```

Figure 36: Accessing RDS through App Instance (Private IP Address)

S3 Storage Configuration

Requirement: Application instances must download from S3 without storing access keys in code.

Method: Access a public S3 bucket via HTTPS URL using curl (no AWS credentials required).

Upload a file (index.html) into S3 Bucket and uniquely name it “s3-bucket-egl211”.

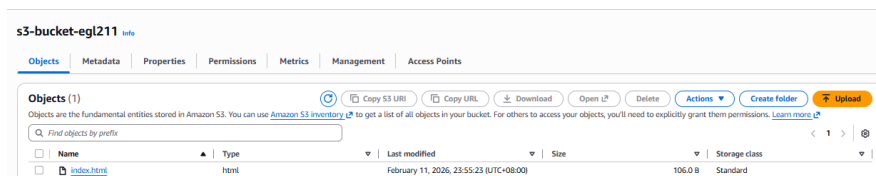
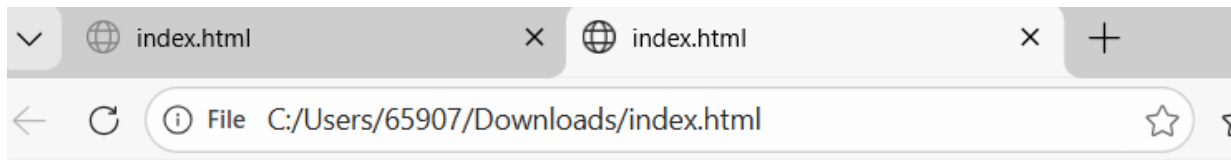


Figure 37: S3 Bucket Objects Uploaded (index.html)



egl211 s3 test

this page was loaded from s3

Figure 38: Contents of index.html file inserted into S3 Bucket

S3 Bucket Policy:

```
{
  "Sid": "DenyInsecureTransport",
  "Effect": "Deny",
  "Principal": "",
  "Action": "s3:",
  "Resource": [
    "arn:aws:s3::: s3-bucket-egl211",
    "arn:aws:s3::: s3-bucket-egl211/"
  ],
  "Condition": {
    "Bool": {
      "aws:SecureTransport": "false"
    }
  }
},
{
  "Sid": "DenyDeleteObject",
  "Effect": "Deny",
  "Principal": "",
  "Action": "s3:DeleteObject",
  "Resource": "arn:aws:s3::: s3-bucket-egl211/"
},
```

```
{
  "Sid": "AllowPublicUpload",
  "Effect": "Allow",
  "Principal": "",
  "Action": "s3:PutObject",
  "Resource": "arn:aws:s3::: s3-bucket-egl211/*"
}
]
```

1. DenyInsecureTransport

This statement denies all S3 actions if the request is not made over HTTPS.

Explanation:

- "Effect": "Deny" blocks access.
- "Action": "s3:*" applies to all S3 operations.
- "Condition": { "aws:SecureTransport": "false" } means any HTTP (non-HTTPS) request is rejected.
- This ensures encrypted communication between client and S3.

2. DenyDeleteObject

This statement prevents objects in the bucket from being deleted.

Explanation:

- "Effect": "Deny" overrides any allow permissions.
- "Action": "s3:DeleteObject" blocks delete operations.
- Protects stored files from accidental or malicious removal.

3. AllowPublicUpload

This statement allows users to upload objects to the bucket.

Explanation:

- "Effect": "Allow" grants permission.
- "Action": "s3:PutObject" allows uploading files.
- "Resource": "arn:aws:s3:::egl211-cas-bucket/*" applies to all objects in the bucket.
- This enables public upload functionality.

Production Recommendation

In production with proper permissions:

1. Create IAM Role with S3 read access
2. Attach role to EC2 instances via Launch Template
3. Use aws s3 cp commands (credentials managed automatically by role)

EBS Snapshots

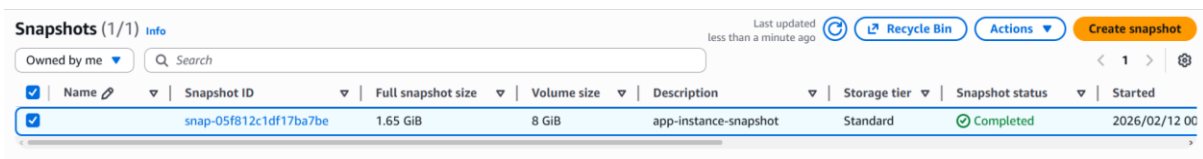
Purpose: This provides backups to ensure business continuity and disaster recovery.

Creating a Snapshot:

1. Go to Instances in the EC2 console
2. Choose one of the App instances and go to the storage tab
3. Press the volume ID link and select the volume
4. Click Actions and Create Snapshot
5. Add tag: Name = "app-snaoshot"
6. Create Snapshot

Use Cases:

- This helps to recover data if the instance was corrupted
- Allows us to launch a new instance from the snapshot
- Allows for cross-region disaster recovery
- It is compliant and meets audit requirements



Name	Snapshot ID	Full snapshot size	Volume size	Description	Storage tier	Snapshot status	Started
app-instance-snapshot	snap-05f812c1df17ba7be	1.65 GiB	8 GiB	app-instance-snapshot	Standard	Completed	2026/02/12 00:00

Figure 39: Snapshot made

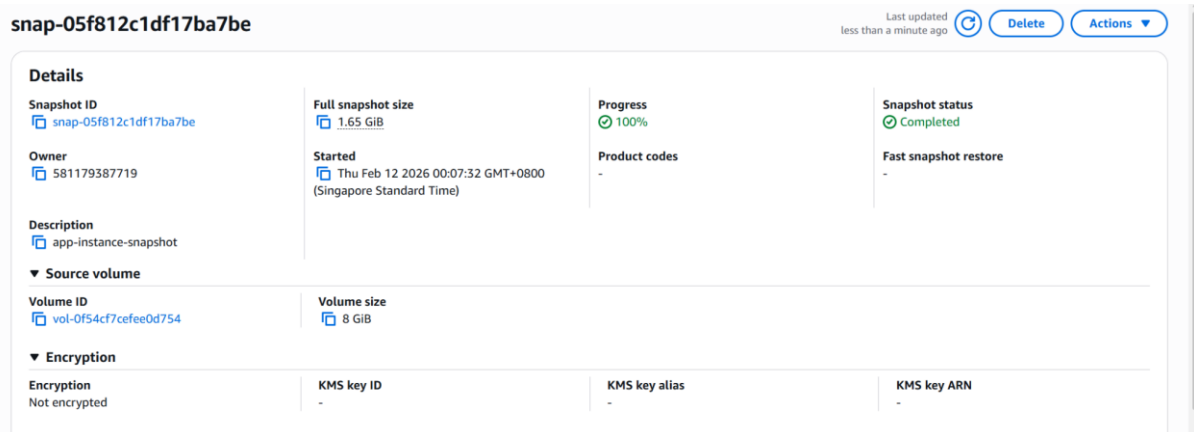


Figure 40: Details of the Snapshot that was created

CloudWatch Monitoring

Purpose: CloudWatch alarms provide proactive monitoring and alerting for infrastructure health

ALB Unhealthy Host Alarm

Configuration:

- Metric: UnHealthyHostCount
- Namespace: AWS/ApplicationELB
- Condition: Greater than 0
- Period: 1 minute

Procedure:

1. Go to CloudWatch Console → Alarms → Create Alarm
2. Select metric → ApplicationELB → Per AppELB, per TG metrics
3. Select UnHealthyHostCount for App-ALB and App-TG
4. Statistic: Average, Period: 1 minute
5. Condition: Greater than 0
6. Configure notification → Alarm Name: "alb-unhealthy-host-alarm"
7. Create alarm

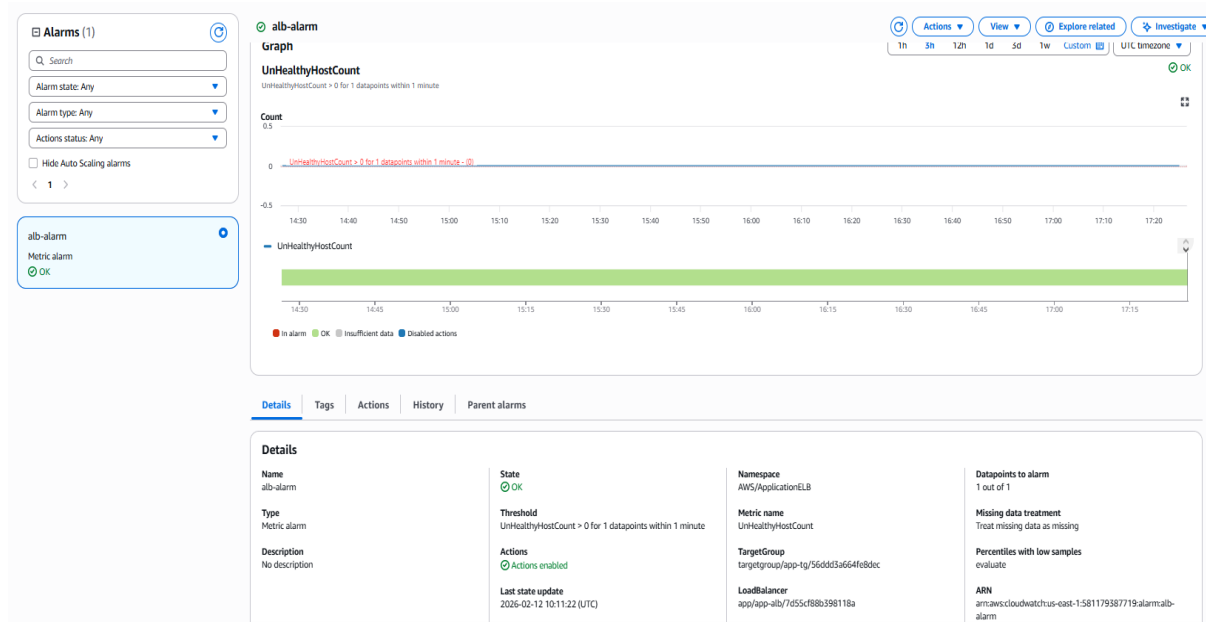


Figure 41: CloudWatch Alarm Configurations

Testing and Validation

Web Application Access

Test: Access web page via ALLB DNS name

Procedure:

- Open browser → <http://<ALB-DNS-NAME>>

Expected Results: Web page displays Instance ID and Availability Zone

Verification: Refresh multiple times - Instance ID changes (proves load balancing)

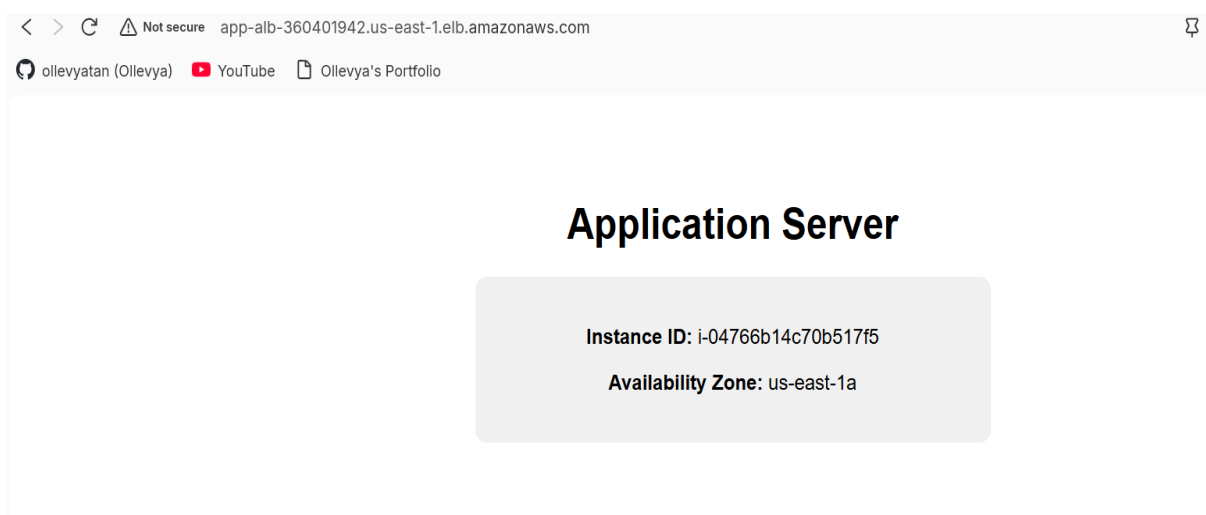


Figure 42: Web server showing what instance is running and which availability zone it is in

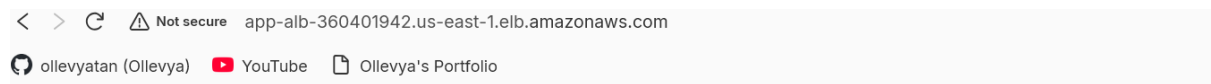
Successfully initiated stopping of i-04766b14c70b517f5

Instances (1/3) [info](#) Last updated 1 minute ago [Connect](#) [Instance state](#) [Actions](#) [La](#)

Find Instance by attribute or tag (case-sensitive) [All states](#)

	Name	Instance ID	Instance state	Instance type	Status check	Alarm status	Availability Zone	Public IPv4 DNS	Public IPv4 ...	Elastic IP	IPv6 IPs	Monitoring	Secur
☐	bastion host	i-0850620cef762c292	Running	t2.micro	2/2 checks passed	View alarms	us-east-1a	-	54.87.115.232	-	-	disabled	basti
☒		i-04766b14c70b517f5	Stopped	t2.micro	-	View alarms	us-east-1a	-	-	-	-	disabled	app-s
☐		i-057a2301be76ef3cf	Running	t2.micro	2/2 checks passed	View alarms	us-east-1b	-	-	-	-	disabled	app-s

Figure 43: Shows that when one instance is stopped



Application Server

Instance ID: i-057a2301be76ef3cf

Availability Zone: us-east-1b

Figure 44: Web server after instance was stopped, shows that load balancer is working because it shows a different instance in a different AZ

High Availability Test

Test: Stop one instance, verify traffic continues

Procedure:

1. Note current Instance IDs from web page
2. EC2 Console → Stop one App-Instance
3. Refresh web page

Expected Result: Page still loads, shows only remaining instance

Auto Scaling Self-Healing

Test: Terminate instance, verify ASG replaces it Procedure:

1. Terminate 1 App Instance
2. Wait 2 - 3 minutes
3. Check EC2 instances and ASG activity tab

Expected Result: New Instance launched automatically

Auto Scaling groups (1/1) Info

Last updated 9 minutes ago

Launch configurations Launch templates Actions

Search your Auto Scaling groups

Name	Launch template/configuration	Instances	Status	Desired capacity	Min	Max	Availability Zones	Creation time
app-asp	app-launch-template Version Default	2	-	2	2	4	2 Availability Zones	Thu Feb 12 2026 00:01:04 GMT+0800 (Singapore Standard Time)

Auto Scaling group: app-asp

Details Integrations Automatic scaling **Instance management** Instance refresh Activity Monitoring Tags - moved

Instances (3)

Filter instances

Instance ID	Lifecycle	Instance type	Weighted capacity	Launch template/configuration	Availability Zone	Health status	Protected from
i-04766b14c70b517f5	Terminating	t2.micro	-	app-launch-template Version 1	use1-az6 (us-east-1a)	Unhealthy	
i-057a2301be76ef3cf	InService	t2.micro	-	app-launch-template Version 1	use1-az1 (us-east-1b)	Healthy	
i-0db8a44ed8a0e8c24	InService	t2.micro	-	app-launch-template Version 1	use1-az6 (us-east-1a)	Healthy	

Figure 45: One instance was terminated and it shows an "Unhealthy" health status

Activity history (16)

Filter activity history

Status	Description	Cause	Start time	End time
Successful	Launching a new EC2 instance: i-0db8a44ed8a0e8c24	At 2026-02-12T18:06:11Z an instance was launched in response to an unhealthy instance needing to be replaced.	2026 February 13, 02:06:13 AM +08:00	2026 February 13, 02:06:18 AM +08:00
Connection draining in progress	Terminating EC2 instance: i-04766b14c70b517f5 - Waiting For ELB Connection Draining.	At 2026-02-12T18:06:11Z an instance was taken out of service in response to an EC2 health check indicating it has been terminated or stopped.	2026 February 13, 02:06:11 AM +08:00	

Figure 46: The Activity History shows that a new instance was launched successfully

Successfully initiated stopping of i-04766b14c70b517f5

Instances (4) Info

Last updated less than a minute ago

Connect Instance state Actions Launch instance

Find Instance by attribute or tag (case-sensitive) All states

Name	Instance ID	Instance state	Instance type	Status check	Alarm status	Availability Zone	Public IPv4 DNS	Public IPv4 ...	Elastic IP	IPv6 IPs	Monitoring	Security group na
bastion host	i-0850620cef762c292	Running	t2.micro	2/2 checks passed	View alarms +	us-east-1a	-	54.87.115.232	-	-	disabled	bastion-sg
	i-04766b14c70b517f5	Stopped	t2.micro	-	View alarms +	us-east-1a	-	-	-	-	disabled	app-sg
	i-0db8a44ed8a0e8c24	Running	t2.micro	2/2 checks passed	View alarms +	us-east-1a	-	-	-	-	disabled	app-sg
	i-057a2301be76ef3cf	Running	t2.micro	2/2 checks passed	View alarms +	us-east-1b	-	-	-	-	disabled	app-sg

Figure 47: This shows the old instance that was terminated and the new instance running, always showing that there's constantly 2 app instances

Database Not Publicly Accessible

Test: Attempt a connection from Bastion instance in PuTTY

Procedure: From local:

```
mysql -h <RDS-ENDPOINT> -u admin -p
```

Expected Result: Connection timeout which proves that the DB is not accessible through public IP.

```
ec2-user@ip-10-0-1-91:~  
Using username "ec2-user".  
Authenticating with public key "imported-openssh-key" from agent  
#  
_##### Amazon Linux 2023  
~~\#####\  
~~\#####\  
~~\###|  
~~\#/ https://aws.amazon.com/linux/amazon-linux-2023  
~~V~'-'->  
~~~~  
~~~~.  
~~/_m/'-/
```

Last login: Fri Feb 13 05:38:20 2026 from 210.10.76.120
[ec2-user@ip-10-0-1-91 ~]\$ mysql -h proj-mysql-db.cznzajrpjxx.us-east-1.rds.amazonaws.com -u admin -p
Enter password:
ERROR 2002 (HY000): Can't connect to MySQL server on 'proj-mysql-db.cznzajrpjxx.us-east-1.rds.amazonaws.com' (115)
[ec2-user@ip-10-0-1-91 ~]\$ █

Figure 50: Attempted connection to RDS from Bastion failed as Bastion uses public IP

Private Instance Internet Access

Test: Verify that NAT Gateway is working

Procedure: From app instance

```
curl -I https://google.com
```

Expected Result: HTTP response received -proves private instances can reach the internet

S3 Testing

To see if S3 can be downloaded without any stored credentials in the instance.

Test Command (from App Instance in PuTTY):

```
curl -O https://bucket-proj-egl211.s3.amazonaws.com/index.html  
cat index.html
```

What this proves:

- App instance in private subnet can reach the internet (NAT Gateway working)
- Can download from S3 without any credentials stored on the instance
- Demonstrates the architecture pattern (in production, use IAM roles)

```
[ec2-user@ip-10-0-1-91 ~]$ ssh ec2-user@10.0.10.166
#_
~\##### Amazon Linux 2023
~~\#####\
~~\#####|
~~\##/\
~~V~'-'> https://aws.amazon.com/linux/amazon-linux-2023
~~
~~
~~. _ /
~/m/' /
Last login: Thu Feb 12 17:47:31 2026 from 10.0.1.91
[ec2-user@ip-10-0-10-166 ~]$ curl -O https://bucket-proj-egl211.s3.amazonaws.com/index.html
% Total    % Received % Xferd Average Speed   Time    Time     Time  Current
           Dload Upload   Total   Spent    Left   Speed
100 106 100 106 0 0 1975 0 --:--:-- --:--:-- --:--:-- 2000
[ec2-user@ip-10-0-10-166 ~]$
```

Figure 51: Curl S3 Bucket in Private App instance

```
bucket-proj-egl211
```

```
[ec2-user@ip-10-0-10-166 ~]$ cat index.html
<html>
  <body>
    <h1>egl211 s3 test</h1>
    <p>this page was loaded from s3</p>
  </body>
</html>[ec2-user@ip-10-0-10-166 ~]$
```

Figure 52: Reading `index.html` file and displaying its contents

In Private EC2 instance, try:

```
aws s3 ls s3://bucket-proj-egl211
```

Expected output: Unable to load credentials

It shows that no IAM role is attached and no access keys are configured.

Problem & Challenges

IAM Role Creation Restrictions

Problems: AWS lab account did not permit IAM role creation.

Impact: Could not implement IAM role for S3 access (best practice).

Solution: Used public S3 bucket with HTTP URL access to demonstrate connectivity.

In production, IAM roles should be used for secure, credential-free access.

EC2 Metadata Services (IMDSv2)

Problem: Initial webpage showed blank Instance ID and Availability Zone.

Cause: Amazon Linux 2023 uses IMDSv2 by default, requiring token-based authentication for metadata requests.

Solution: Updated user data script to:

1. First obtain a token using PUT request
2. Use token in subsequent metadata API calls

Original (failed):

```
INSTANCE_ID=$(curl -s http://169.254.169.254/latest/meta-data/instance-id)
```

Fixed:

```
TOKEN=$(curl -X PUT "http://169.254.169.254/latest/api/token" -H aws-ec2-  
metadata-token-ttl-seconds: 21600")  
INSTANCE_ID=$(curl -s -H "X-aws-ec2-metadata-token: $TOKEN"  
http://169.254.169.254/latest/meta-data/instance-id)
```

Bastion Host IP Change

Problem: Could not SSH to Bastion after stopping / starting instance

Cause: EC2 instances get new public IP when restarted unless using Elastic IP.

Solution: Check EC2 console for the new public IP after every restart. We attach the Elastic IP to Bastion for persistent addressing which is for production purposes.

Individual Reflections

Yadanar's Reflection

Implementing and validating the Database Tier helped me better understand how to balance High Availability with security. Deploying an AWS RDS Multi-AZ instance was important because it provides automatic synchronous replication to a standby instance and promotes it quickly if the primary fails. This showed me how cloud architecture supports resilience beyond basic deployment. At the same time, disabling public access reinforced the importance of keeping the database fully private and accessible only through controlled internal layers. I also learned the significance of the RDS DNS endpoint, which remains unchanged during failover, allowing the application to stay connected without requiring configuration changes.

From a security perspective, I applied the Principle of Least Privilege to ensure that traffic flowed strictly through the intended architecture path. Testing connectivity was an important learning point. When attempting to connect from the Bastion Host, the connection correctly failed, confirming that the database was not directly accessible. However, connecting from the App Instance via the Bastion was successful, validating that the trusted access path was working as designed. Overall, this phase strengthened my understanding of how managed databases can provide both reliability and security within a well-architected cloud environment.

Ollevya's Reflection

During the implementation of the bastion host architecture, I gained a deeper understanding of how AWS networking and security controls work together to reduce attack surface. Initially, I underestimated the impact of security group source restrictions and user data scripts on connectivity and system behaviour. Through troubleshooting SSH timeouts and MySQL connection delays, I learned how package

management processes and NAT Gateway performance can affect instance availability.

This task reinforced the importance of the principle of least privilege, particularly in designing security groups that allow access only from trusted sources rather than open IP ranges. I also developed a clearer distinction between administrative access (bastion host) and application access, which is critical in real-world cloud environments.

Overall, this exercise improved my confidence in designing secure, highly available AWS architectures and highlighted the value of deliberate planning before deployment to avoid unnecessary rework.

Min's Reflection

Working on the load balancing part helped me understand more clearly how AWS keeps applications stable and available. By setting up the Application Load Balancer together with the Target Group and Auto Scaling Group, I learned how incoming traffic can be shared across multiple servers instead of depending on just one. When I tested it by refreshing the ALB page and seeing different instance IDs appear, it made the concept feel very real and practical. I also learned that health check settings are very important, because if they are not configured properly, instances can be marked unhealthy and won't receive traffic. Troubleshooting this taught me how routing and monitoring work together. Overall, this task gave me a better hands-on understanding of high availability and scalability in a simple and practical way.

Zilds's Reflection

During this project, I took ownership of the data protection strategy and the technical integration of our tiered architecture. Implementing Requirement 4.4 through EBS Snapshots taught me the practical application of Point-in-Time recovery; by capturing a point-in-time image of our application volume, I ensured that we had a reliable recovery point to mitigate risks like data corruption or accidental deletion. This highlighted the importance of "Business Continuity" in a professional Associate Engineer role, moving beyond simple deployment into long-term infrastructure stability.

Beyond my specific tasks, I acted as a technical lead for the group's troubleshooting efforts. I guided team members through the complexities of Step 10 (RDS Deployment) and verified the multi-step SSH connection paths required to reach our private resources via the Bastion Host. A significant challenge was ensuring everyone understood the "jump" logic—moving from the local terminal to the Bastion, then to the App or DB layers—which solidified my understanding of secure administrative access.

To further safeguard our progress, I maintained a backup project configuration (referenced in our VPC and security checklists) to ensure we had a functional baseline in case of configuration drift. Overall, this phase strengthened my ability to not only build cloud components but also to debug the networking "glue" that allows them to function securely as a cohesive system.

Angelin's Reflection

During this project, I learned how to manage teams and gained a deeper understanding of Cloud in general. The difficulty of working in a team whereby the software could not be shared really posed as a challenge to all of us. We had to constantly be aware of what each other's tasks were and whether they were completed. Because there are so many ways to deploy systems in AWS, the struggle for me was that everyone's methods were right and it caused a lot of confusion.

This just proves the importance of communication between team members and the importance of updating without needing to ask. Apart from that, I really gained a lot of understanding on implementation of VPCs and subnets and how they should be routed. As well as EC2 instances and why some may need to have private IPs only and not accessible through the internet. This project has taught me a lot of critical thinking skills, and my brain was tested to its maximum capacity when going through troubleshooting. Troubleshooting problems like being unable to connect to PuTTY happened often. Whether there was a problem with security group inbound rules or a problem with the app instance user data, it was not an easy feat to troubleshoot them. But nonetheless, a lot of valuable lessons were learned, and I look forward to doing the next project.

Brandon's Reflection

During this task, I learned that setting up cloud storage is not just creating a bucket and upload file only. At first, I thought it was very simple, but when doing the bucket policy and testing access, I realised security settings are very important and

sometimes can block myself also if too strict. I also faced some confusion between public and private download, and I learned how preassigned URL works to give temporary access without saving access keys inside EC2.

I also understand more about least privilege and why we should only allow the actions we really need. Sometimes the AWS lab limitations made things harder, especially IAM role and permissions, so I had to find alternative ways to make it work. Overall I learned how different AWS services connect and how small mistakes in policy can affect the whole system. This project helped me better understand practical cloud security and troubleshooting step by step.

Recommendations

All recommendations were researched through reading through AWS technical documentation.

Security Enhancements (AWS, 2024)

1. **AWS WAF:** Implement Web Application Firewall on ALB to protect against SQL injection, XSS, and other web exploits.
2. **VPC Flow Logs:** Enable flow logs for network traffic monitoring and security analysis (requires IAM permissions).
3. **AWS CloudTrail:** Enable API activity logging for auditing and compliance.
4. **Restrict Bastion SSH:** Change Bastion-SG inbound rule from 0.0.0.0/0 to specific administrator IP addresses.
5. **Secrets Manager:** Use AWS Secrets Manager for database credentials instead of hardcoded passwords.

Cost Optimization (AWS,2024)

1. **Reserved Instances:** For predictable workloads, purchase Reserved Instances for significant cost savings.
2. **Right-sizing:** Monitor instance utilization and adjust instance types accordingly.
3. **NAT Gateway Alternatives:** Consider NAT instances for development environments to reduce costs.

Operational Excellence (AWS, 2024)

1. **Infrastructure as Code:** Use CloudFormation or Terraform for reproducible, version-controlled deployments.
2. **CI/CD Pipelines:** Implement automated deployment pipelines for application updates.
3. **Comprehensive Monitoring:** Implement CloudWatch dashboards and additional alarms for proactive monitoring.
4. **Backup Strategy:** Implement automated RDS snapshots and cross-region backup replication.

Conclusions

In this project we successfully implemented a 3-tier architecture on AWS which is high available and meets all the requirements that were needed.

- Creating a custom VPC with 6 subnets across 2 AZs with proper routing towards IGW and NAT GW.
- Using ALB to distribute traffic across ASG-managed instances and includes health checks.
- Deployed a multi-AZ RDS using MySQL that is not publicly accessible by only allowing app-sg to access port 3306 in inbound rules.
- Created Bastion Host to ensure secure SSH access to private instances.
- Deployed EBS Snapshots to ensure business continuity

- Able to download S3 without stored credentials from private instances
- Deployed in-depth defence systems through security groups, network ACLs, and CloudWatch monitoring.

Through this project, we were able to deepen our understanding of AWS networking concepts and implement high availability across multiple AZs. We also learned about including least privilege as one of the best security practices. Implementing auto scaling to improve resilience and cost optimisation and have managed database services for efficiency. We learned that with a good architecture, cloud infrastructure can automatically recover from failures, be scaled to meet demands and maintain security without needing human interventions.

References

Rance, K. (2024). *The rise of cloud Computing: Transforming IT infrastructure*.

83Zero. <https://www.83zero.com/blogs/the-rise-of-cloud-computing-transforming-it-infrastructure/>

Gridelli, S. (2025, 29 January). *What is an RFC 1918 Address?* Netbeez.

<https://netbeez.net/blog/rfc1918/>

Senarath, U. S. (2025, November 4). *Understanding subnets in Amazon VPC: The Foundation of AWS Networking*. Medium.

<https://medium.com/@udarasenarath/understanding-subnets-in-amazon-vpc-the-foundation-of-aws-networking-a3e4c252c6cc>

Amazon Web Services. (2024). *Security group rules*.

https://docs.aws.amazon.com/vpc/latest/userguide/VPC_SecurityGroups.html

Amazon Web Services. (2024). *AWS CloudTrail user guide*.

<https://docs.aws.amazon.com/awsccloudtrail/latest/userguide/>

Amazon Web Services. (2024). *VPC flow logs*.

<https://docs.aws.amazon.com/vpc/latest/userguide/flow-logs.html>

Amazon Web Services. (2024). *AWS WAF developer guide*.

<https://docs.aws.amazon.com/waf/>

Amazon Web Services. (2024). *AWS Secrets Manager user guide*.

<https://docs.aws.amazon.com/secretsmanager/latest/userguide>

Amazon Web Services. (2024). *Reserved instances*.

<https://aws.amazon.com/ec2/pricing/reserved-instances/>

sg-079025d16150be9e5 - alb-sg2

Details | **Inbound rules** | Outbound rules | Sharing | VPC associations | Related resources - new | Tags

Inbound rules (2)

Q Search

Manage tags Edit inbound rules

☐	Name	Security group rule ID	IP version	Type	Protocol	Port range	Source	Description
☐	-	sg-0672f67494cb1e411	IPv4	HTTPS	TCP	443	0.0.0.0/0	-
☐	-	sg-021af7c476586e435	IPv4	HTTP	TCP	80	0.0.0.0/0	-

Figure 54: Inbound rules for alb-sg

sg-079025d16150be9e5 - alb-sg2

Details | Inbound rules | **Outbound rules** | Sharing | VPC associations | Related resources - new | Tags

Outbound rules (1)

Q Search

Manage tags Edit outbound rules

☐	Name	Security group rule ID	IP version	Type	Protocol	Port range	Destination	Description
☐	-	sg-02556282152c27f58	IPv4	All traffic	All	All	0.0.0.0/0	-

Figure 55: Outbound rule for alb-sg

sg-00ead9728a9e68907 - app-sg

Details | **Inbound rules** | Outbound rules | Sharing | VPC associations | Related resources - new | Tags

Inbound rules (2)

Q Search

Manage tags Edit inbound rules

☐	Name	Security group rule ID	IP version	Type	Protocol	Port range	Source	Description
☐	-	sg-052b211d98ff674f9	-	SSH	TCP	22	sg-079014cf1b09b5c64 / bastion-sg	-
☐	-	sg-0f46086755119db7b	-	HTTP	TCP	80	sg-079025d16150be9e5 / alb-sg2	-

Figure 56: Inbound rules for app-sg

sg-00ead9728a9e68907 - app-sg

Details | Inbound rules | **Outbound rules** | Sharing | VPC associations | Related resources - new | Tags

Outbound rules (1)

Q Search

Manage tags Edit outbound rules

☐	Name	Security group rule ID	IP version	Type	Protocol	Port range	Destination	Description
☐	-	sg-00fa49493db0c02bc	IPv4	All traffic	All	All	0.0.0.0/0	-

Figure 57: Outbound rules for app-sg

sg-079014cf1b09b5c64 - bastion-sg

Details | **Inbound rules** | Outbound rules | Sharing | VPC associations | Related resources - new | Tags

Inbound rules (1)

Q Search

Manage tags Edit inbound rules

☐	Name	Security group rule ID	IP version	Type	Protocol	Port range	Source	Description
☐	-	sg-0abeb9e7418cfa052	IPv4	SSH	TCP	22	0.0.0.0/0	-

Figure 58: Inbound rules for bastion-sg

sg-079014c1b09b5c64 - bastion-sg

Details | Inbound rules | **Outbound rules** | Sharing | VPC associations | Related resources - new | Tags

Outbound rules (1)

Q Search

☐	Name	Security group rule ID	IP version	Type	Protocol	Port range	Destination	Description
☐	-	sg-08c2ba329fc4bdea	IPv4	All traffic	All	All	0.0.0.0/0	-

Manage tags Edit outbound rules

Figure 59: Outbound Rules for bastion-sg

sg-04ebd2b459b31240e - db-sg

Details | **Inbound rules** | Outbound rules | Sharing | VPC associations | Related resources - new | Tags

Inbound rules (1)

Q Search

☐	Name	Security group rule ID	IP version	Type	Protocol	Port range	Source	Description
☐	-	sg-0f45b853d5Qu1115	-	MySQL/Aurora	TCP	3306	sg-00eac972ba9c58907 / sg-sg	-

Manage tags Edit inbound rules

Figure 60: Inbound rules for db-sg

sg-04ebd2b459b31240e - db-sg

Details | Inbound rules | **Outbound rules** | Sharing | VPC associations | Related resources - new | Tags

Outbound rules (1)

Q Search

☐	Name	Security group rule ID	IP version	Type	Protocol	Port range	Destination	Description
☐	-	sg-0f45b853d5Qu1115	IPv4	All traffic	All	All	0.0.0.0/0	-

Manage tags Edit outbound rules

Figure 61: Outbound rules for db-sg

acl-014b48f775fd1563e / private-nacl

Details | **Inbound rules** | Outbound rules | Subnet associations | Tags

Inbound rules (3)

Q Filter inbound rules

Rule number	Type	Protocol	Port range	Source	Allow/Deny
100	HTTP (80)	TCP (6)	80	10.0.0.0/16	Allow
110	SSH (22)	TCP (6)	22	10.0.0.0/16	Allow
*	All traffic	All	All	0.0.0.0/0	Deny

Edit inbound rules

Figure 62: Inbound rules for private-nacl

acl-014b48f775fd1563e / private-nacl

Details | Inbound rules | **Outbound rules** | Subnet associations | Tags

Outbound rules (3)

Q Filter outbound rules

Rule number	Type	Protocol	Port range	Destination	Allow/Deny
100	HTTP (80)	TCP (6)	80	0.0.0.0/0	Allow
110	HTTPS (443)	TCP (6)	443	0.0.0.0/0	Allow
*	All traffic	All	All	0.0.0.0/0	Deny

Edit outbound rules

Figure 63: Outbound rules for private-nacl

ac1-014b48f775d1563e / private-nacl

Details Inbound rules Outbound rules Subnet associations Tags

Subnet associations (6) [Edit subnet associations](#)

Q Filter subnet associations

Name	Subnet ID	Associated with	Availability Zone	IPv4 CIDR	IPv6 CIDR
pub-sub-1a	subnet-07f70c39312229f68	ac1-014b48f775d1563e / private-nacl	us-east-1a	10.0.1.0/24	-
pub-sub-1b	subnet-057e12979339126d8	ac1-014b48f775d1563e / private-nacl	us-east-1b	10.0.2.0/24	-
priv-db-sub-1a	subnet-07f697c314806ac37	ac1-014b48f775d1563e / private-nacl	us-east-1a	10.0.100.0/24	-
priv-app-sub-1a	subnet-054612cc667c0345b	ac1-014b48f775d1563e / private-nacl	us-east-1a	10.0.10.0/24	-
priv-db-sub-1b	subnet-0b0530c95c4605e	ac1-014b48f775d1563e / private-nacl	us-east-1b	10.0.200.0/24	-
priv-app-sub-1b	subnet-01a57959c30a81f7	ac1-014b48f775d1563e / private-nacl	us-east-1b	10.0.20.0/24	-

Figure 64: subnet associations for private-nacl

rtb-0aaa879f117ea9d4d / pub-rtb [Actions](#)

Details [Info](#)

Route table ID [rtb-0aaa879f117ea9d4d](#)

VPC [vpc-08545f72260d32a53](#) | [proj-cas](#)

Main [No](#)

Owner ID [802286170681](#)

Explicit subnet associations [2 subnets](#)

Edge associations [-](#)

Routes Subnet associations Edge associations Route propagation Tags

Routes (2) [Both](#) [Edit routes](#)

Q Filter routes

Destination	Target	Status	Propagated
0.0.0.0/0	sgw-04f245ca6939f5554	Active	No
10.0.0.0/16	local	Active	No

Figure 65: public route table routes

rtb-0aaa879f117ea9d4d / pub-rtb [Actions](#)

Details [Info](#)

Route table ID [rtb-0aaa879f117ea9d4d](#)

VPC [vpc-08545f72260d32a53](#) | [proj-cas](#)

Main [No](#)

Owner ID [802286170681](#)

Explicit subnet associations [2 subnets](#)

Edge associations [-](#)

Routes Subnet associations Edge associations Route propagation Tags

Explicit subnet associations (2) [Edit subnet associations](#)

Q Find subnet association

Name	Subnet ID	IPv4 CIDR	IPv6 CIDR
pub-sub-1b	subnet-057e12979339126d8	10.0.2.0/24	-
pub-sub-1a	subnet-07f70c39312229f68	10.0.1.0/24	-

Figure 66: Public Route table subnet associations

rtb-0cafb211d4716862 / priv-rtb1a [Actions](#)

Details [Info](#)

Route table ID [rtb-0cafb211d4716862](#)

VPC [vpc-08545f72260d32a53](#) | [proj-cas](#)

Main [No](#)

Owner ID [802286170681](#)

Explicit subnet associations [2 subnets](#)

Edge associations [-](#)

Routes Subnet associations Edge associations Route propagation Tags

Routes (2) [Both](#) [Edit routes](#)

Q Filter routes

Destination	Target	Status	Propagated
0.0.0.0/0	nat-1129f464d18578899	Active	No
10.0.0.0/16	local	Active	No

Figure 67: Private route table us-east-1a routes

rtb-0cafb211d4716862 / priv-rtb1a

Actions

Details

Route table ID

rtb-0cafb211d4716862

VPC

vpc-08545f72260d32a53 | proj-cas

Main

No

Owner ID

802286170681

Explicit subnet associations

2 subnets

Edge associations

-

Routes

Subnet associations

Edge associations

Route propagation

Tags

Explicit subnet associations (2)

Edit subnet associations

Find subnet association

Name

Subnet ID

IPv4 CIDR

IPv6 CIDR

priv-app-sub-1a

subnet-054612cc567c0345b

10.0.10.0/24

-

priv-db-sub-1a

subnet-07b97c514806bc97

10.0.100.0/24

-

Figure 68: Private route table us-east-1a subnet associations

rtb-0d9a8ae82fd6c2fb3 / priv-rtb-1b

Actions

Details

Route table ID

rtb-0d9a8ae82fd6c2fb3

VPC

vpc-08545f72260d32a53 | proj-cas

Main

No

Owner ID

802286170681

Explicit subnet associations

2 subnets

Edge associations

-

Routes

Subnet associations

Edge associations

Route propagation

Tags

Routes (2)

Both

Edit routes

Filter routes

Destination

Target

Status

Propagated

0.0.0.0/0

nat-16f6a9151b13ec784

Active

No

10.0.0.0/16

local

Active

No

Figure 69: Private route table us-east-1b routes

rtb-0d9a8ae82fd6c2fb3 / priv-rtb-1b

Actions

Details

Route table ID

rtb-0d9a8ae82fd6c2fb3

VPC

vpc-08545f72260d32a53 | proj-cas

Main

No

Owner ID

802286170681

Explicit subnet associations

2 subnets

Edge associations

-

Routes

Subnet associations

Edge associations

Route propagation

Tags

Explicit subnet associations (2)

Edit subnet associations

Find subnet association

Name

Subnet ID

IPv4 CIDR

IPv6 CIDR

priv-db-sub-1b

subnet-0bd0530c95cfd605e

10.0.200.0/24

-

priv-app-sub-1b

subnet-01a579399c30b61f7

10.0.20.0/24

-

Figure 70: private route table use-east-1b subnet associations